

Laboratorio I

a.a. 2025/2026

Esercitazione su TypeScript

Esercizio 1: Traduzione js → ts

Questo esercizio va svolto in **TypeScript**. Si consideri il seguente esercizio, già svolto in JavaScript nelle esercitazioni precedenti. Si richiede di **riscriverlo in TypeScript**, annotando **nel modo più preciso possibile**: campi, parametri, valori di ritorno, strutture dati, eccezioni, metodi statici. Non usare **any**.

Si implementi una classe **Parcometro** che rappresenta un parcometro personale con credito protetto. Il costruttore prende due argomenti: **targa**, una stringa non vuota; **credito**, un numero non negativo (default **0**). Se i parametri non sono validi, il costruttore deve lanciare un'eccezione di tipo **TypeError**. La classe deve consentire l'accesso in sola lettura al credito corrente disponibile e offrire i seguenti metodi: **ricarica(euro, motivo)** che aggiunge **euro** al credito; **paga(euro, motivo)** che sottrae **euro** dal credito. I metodi **ricarica** e **paga** devono lanciare **TypeError** se **euro** non è un numero strettamente positivo, oppure **motivo** non è una stringa. Il metodo **paga** deve inoltre lanciare un'eccezione custom **CreditoInsufficiente** se **euro** è maggiore del credito disponibile. I metodi **ricarica** e **paga** devono istanziare oggetti della classe **Movimento** con le seguenti proprietà: **tipo** ("R" = ricarica, "S" = spesa), **euro**, **motivo**, **data**. **Parcometro** fornisce inoltre il metodo **storico(k)** che restituisce un array con gli ultimi **k** movimenti effettuati (default **k = 5**), ordinati dal più recente al meno recente. Attenzione: modificare gli oggetti restituiti da **storico()** non deve alterare i movimenti interni al parcometro. Ogni operazione effettuata da una istanza di **Parcometro** deve inoltre essere registrata in un **registro globale della classe**. Ogni elemento del registro globale è un oggetto con due proprietà: **parcometro**, che contiene il riferimento all'istanza di **Parcometro**; **movimento**, che contiene il riferimento al corrispondente **Movimento**. La classe deve offrire un metodo statico **registro()** che restituisce un array con tutte le operazioni effettuate su tutte le istanze create. Attenzione: modificare l'array restituito da **registro()** non deve alterare il registro globale della classe. Si curino con particolare attenzione i tipi.

Esercizio 2: Playlist

Questo esercizio va svolto in **TypeScript**. Una traccia musicale è rappresentata da un oggetto con 4 proprietà: **titolo**, **artista** (stringhe) **durata** (numerico), **preferita** (un booleano opzionale).

- Si definisca una interfaccia **Traccia**.
- Si scriva una funzione **durataTotale(lista)** che restituisca la somma delle durate di tutte le tracce in **lista**.
- Si scriva una funzione **soloPreferite(lista)** che restituisca un nuovo array contenente solo le tracce per cui **preferita** vale **true**.
- Si scriva una funzione **descrivi(t)** che restituisca:
 - **"titolo - artista"** se **preferita** non è presente oppure vale **false**;
 - **"titolo - artista ★"** se **preferita** vale **true**.
- Si scriva una funzione **aggiungiGenere(t, genere)** che restituisca un nuovo oggetto con tutte le proprietà di **t** più la proprietà **genere: string**.

Si annotino tutti i tipi nel modo più preciso possibile.

Esercizio 3: Distributore Automatico

Questo esercizio va svolto in **TypeScript**. Un messaggio prodotto da un distributore automatico può essere: una stringa, un numero, null o undefined.

- Si definisca un tipo **Messaggio**.
- Si scriva una funzione **formatta(m)** che:
 - restituisce la stringa stessa se **m** è una stringa;
 - restituisce "**CODICE n**" se **m** è un numero **n**;
 - restituisce "**NESSUN MESSAGGIO**" se **m** è **null** oppure **undefined**.
- Si scriva una funzione **formattaTutti(ms)** che, dato un array di messaggi, restituisca un array di stringhe.
- Si scriva una funzione **leggiValore(x)** che riceve un valore di tipo **unknown** e:
 - se **x** è un valore ammesso per **Messaggio**, lo restituisce;
 - altrimenti lancia una **TypeError**.

Non usare **any**.

Esercizio 4: Robot

Questo esercizio va svolto in **TypeScript**. Una direzione è un valore fra **Nord**, **Sud**, **Est**, **Ovest**. Uno spostamento è un oggetto con proprietà: **d**, che rappresenta la direzione; **l**, che rappresenta la lunghezza dello spostamento.

Una posizione è rappresentata come una tupla di due numeri.

- Si definisca una **enum Dir**.
- Si definisca una interfaccia **Step**.
- Si definisca un tipo **Point** per rappresentare una posizione.
- Si scriva una funzione **walk(o, p)** che, dati una posizione iniziale **o** e un percorso **p**, restituisca la posizione finale raggiunta seguendo tutti gli spostamenti.
- Si assuma che **p** sia rappresentato da un array di spostamenti.
- Si definisca un tipo funzione **Trasformatore** per rappresentare una funzione che riceve una posizione e restituisce o una nuova posizione, o una stringa, o un numero.
- Si scriva una funzione **applica(o, p, f?)** che:
 - calcola la posizione finale di **walk(o,p)**;
 - se **f** è fornita, restituisce **f(posizioneFinale)**;
 - altrimenti restituisce direttamente la posizione finale.

Si annotino tutti i tipi nel modo più preciso possibile.

Esercizio 5: Prenotazioni di missione

Questo esercizio va svolto in **TypeScript**. Si implementi una classe `Passeggero` con: costruttore (`nome`, `categoria`), dove `nome` è una stringa non vuota e `categoria` vale `"adulto"` oppure `"minore"`; getter `nome`; getter `categoria`. Se i parametri non sono validi, il costruttore deve lanciare `TypeError`. Si implementi poi una classe `Prenotazione` con: costruttore (`codice`, `posti`), dove `codice` è una stringa non vuota e `posti` è un intero strettamente positivo; elenco privato dei passeggeri, inizialmente vuoto; stato `"aperta"` oppure `"chiusa"`, inizialmente `"aperta"`. La classe `Prenotazione` deve offrire i metodi: `aggiungi(p)`, che aggiunge il passeggero `p`; `chiudi()`, che chiude la prenotazione; `elenco()`, che restituisce i passeggeri nell'ordine di inserimento; `conteggioMinori()`, che restituisce il numero di passeggeri con categoria `"minore"`; `postiLiberi()`, che restituisce il numero di posti ancora disponibili. Si richiede inoltre che: se la prenotazione è chiusa, `aggiungi` lanci `PrenotazioneChiusaError`; se non ci sono posti liberi, `aggiungi` lanci `PrenotazionePienaError`; se si tenta di aggiungere due volte lo stesso oggetto `Passeggero`, `aggiungi` lanci `PasseggeroDuplicatoError`. Si implementi infine una sottoclasse `PrenotazionePremium` che estende `Prenotazione`, con: costruttore (`codice`, `posti`, `servizioExtra`), dove `servizioExtra` è una stringa non vuota; getter `servizioExtra`. `PrenotazionePremium` deve offrire inoltre un metodo `elencoPremium()` che restituisca un array di oggetti con proprietà `nome` ed `extra`, dove `nome` è il nome del passeggero ed `extra` è il valore di `servizioExtra`. Si scriva infine una funzione `chiudiPrenotazioniPiene(xs)` che: prende un array eterogeneo `xs`; per ogni elemento di `xs` che sia una istanza di `Prenotazione` o `PrenotazionePremium`, chiude la prenotazione se non ha posti liberi; restituisce un nuovo array con tutte e sole le prenotazioni effettivamente chiuse dalla funzione, nello stesso ordine di `xs`. La funzione non deve interrompersi in presenza di elementi di altro tipo. Si annotino tutti i tipi nel modo più preciso possibile.

Q & A